OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Advanced SQL: Grouping, Joins & Set Operations

Aggregate Functions, GROUP BY, HAVING, Relational Joins, Set Theoretic Operators & Subqueries

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 6 (Class 12)	Aggregate Functions: `COUNT(*)`, `COUNT(col)`, `COUNT(DISTINCT col)`, `SUM()`, `AVG()`, `MIN()`, `MAX()`. Handling of NULL values in aggregation.	`COUNT(*)` counting all tuples vs `COUNT(col)` ignoring NULLs, `AVG(col) = SUM / COUNT(col)` dividing strictly by non-null count.
Section B1: Topic 6 (Class 12)	Grouping Data: `GROUP BY` clause, multi-column grouping, filtering groups using `HAVING` clause, execution order.	Non-aggregate columns in `SELECT` not in `GROUP BY` triggering errors, `WHERE` (pre-grouping) vs `HAVING` (post-grouping), NULL grouping.
Section B1: Topic 6 (Class 12)	Joins & Set Operations: `INNER JOIN`, `NATURAL JOIN`, `CROSS JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN`, `UNION`, `UNION ALL`, Subqueries.	Degree and Cardinality bounds of Joins, Natural Join duplicate column removal, Dangling tuple NULL-padding, `UNION` vs `UNION ALL`.

EXAM PATTERN

Advanced SQL questions in CUET Computer Science test execution pipelines and aggregate invariants:

1. **Aggregate NULL Handling:** `COUNT(*)` counts **every row** (even rows where all attributes are NULL). All other aggregate functions (`COUNT(col)`, `SUM`, `AVG`, `MIN`, `MAX`) strictly ignore `NULL`!
2. **The AVG() Invariant:** If a table has 4 rows with values `(10, 20, 30, NULL)`, `AVG(val)` is $\frac{10+20+30}{3} = 20$, **not** $\frac{60}{4} = 15$!
3. **WHERE vs HAVING:** You **cannot** write `WHERE COUNT(*) > 5`! Aggregate conditions must be placed in `HAVING`. Conversely, filter individual rows in `WHERE` before grouping to optimize query performance.
4. **SELECT vs GROUP BY:** Any column present in the `SELECT` list that is not enclosed inside an aggregate function **must** be present in the `GROUP BY` list.
5. **Natural Join Degree:** $\text{Degree}(R \bowtie S) = n_1 + n_2 - k$, where k is the number of shared attributes. Do not duplicate common columns!
6. **UNION vs UNION ALL:** `UNION` sorts and eliminates duplicate rows; `UNION ALL` simply concatenates result sets, preserving duplicates.

Aggregate Functions in SQL

Aggregate Functions (also termed **Multiple-Row** or **Group Functions**) take an entire collection of values across a column or partition as input and return a single summary value. They represent the foundation of business intelligence, reporting, and statistical analytics in relational databases:

The Five Standard Aggregate Functions

1. `COUNT()`: Evaluates the number of rows or non-null attribute values.
2. `SUM()`: Calculates the arithmetic sum of numeric values.
3. `AVG()`: Calculates the arithmetic mean of numeric values.
4. `MIN()`: Identifies the minimum value (numeric magnitude, alphabetical string sort order, or earliest chronological date).
5. `MAX()`: Identifies the maximum value (numeric magnitude, alphabetical string sort order, or latest chronological date).

In-Depth Behavior of MIN() and MAX() Across Data Types

While `SUM()` and `AVG()` strictly require numeric operands, `MIN()` and `MAX()` operate across all data domains:

- **Character Strings:** Evaluated based on ASCII / Unicode collation sequence.
  - `MIN('Aarav', 'Zoya', 'Bhavna')` evaluates to `'Aarav'`.
  - `MAX('Aarav', 'Zoya', 'Bhavna')` evaluates to `'Zoya'`.
- **Temporal (DATE / DATETIME):** Evaluated based on chronological timeline.
  - `MIN(DOB)` returns the **earliest** date (i.e., the oldest individual).
  - `MAX(DOB)` returns the **latest / most recent** date (i.e., the youngest individual).

Distinct Aggregations

Standard SQL allows qualifying column names with `DISTINCT` inside aggregate functions to discard duplicate values before computation:

- `COUNT(DISTINCT Salary)`: Counts only unique salary figures.
- `SUM(DISTINCT Salary)`: Sums distinct salaries, counting identical figures only once.
- `AVG(DISTINCT Salary)`: Evaluates the average of unique salary figures.

Aggregate Expression	Values Evaluated	Returned Output & Semantics
`COUNT(Salary)`	`{50000, 50000, 60000, NULL}`	`3` (Counts all 3 non-null salary entries).
`COUNT(DISTINCT Salary)`	`{50000, 60000}`	`2` (Counts unique non-null salary values).
`SUM(Salary)`	`50000 + 50000 + 60000`	`160000` (Sums all non-null entries).
`SUM(DISTINCT Salary)`	`50000 + 60000`	`110000` (Sums unique values only).
`AVG(Salary)`	`160000 / 3`	`53333.33` (Divides by count of non-null values).
`AVG(DISTINCT Salary)`	`110000 / 2`	`55000.00` (Divides by count of unique values).

The Golden Rule of Aggregate Functions and NULL Values

Understanding how `NULL` values interact with aggregate functions is the single most frequently tested concept in CUET SQL:

Function Syntax	NULL Handling Rule	Mathematical Semantics & Behavior
`COUNT(*)`	**Counts All Rows**	Returns the total count of tuples in the relation/group. **Never ignores NULLs** because it counts rows, not individual column values.
`COUNT(column)`	**Ignores NULLs**	Returns the number of tuples where the specified column contains a non-null value: $\text{COUNT}(A) = \sum_{t \in R, t.A \neq \text{NULL}} 1$.
`COUNT(DISTINCT col)`	**Ignores NULLs & Duplicates**	Returns the count of distinct, non-null values present in the column.
`SUM(column)`	**Ignores NULLs**	Computes the arithmetic sum of non-null values. If all rows in the group are NULL, `SUM()` evaluates to `NULL` (not 0).
`AVG(column)`	**Ignores NULLs**	Computes the average: `AVG(A) = SUM(A) / COUNT(A)`. **Divides strictly by the count of non-null values!**

Function Syntax	NULL Handling Rule	Mathematical Semantics & Behavior
`MIN(col)` / `MAX(col)`	**Ignores NULLs**	Returns the extreme non-null value. If all rows are NULL, returns `NULL`.

AGGREGATE FUNCTION CALCULATIONS WITH NULLS

Problem: Consider the table `EMP_BONUS`:

- E1: (101, 'Arjun', 50000, 5000)
- E2: (102, 'Bhavna', 60000, NULL)
- E3: (103, 'Chetan', 70000, 7000)
- E4: (104, 'Divya', 40000, NULL)

Evaluate the exact numerical result of each query:

1. `SELECT COUNT(*), COUNT(Bonus), COUNT(DISTINCT DeptId) FROM EMP_BONUS;` (Assume DeptIds are 10, 20, 10, 20).
2. `SELECT SUM(Bonus), AVG(Bonus) FROM EMP_BONUS;`

Solution:

- **Part 1:**
  - `COUNT(*)` counts all 4 rows in the table $\rightarrow 4$.
  - `COUNT(Bonus)` counts non-null bonus values. E1 (5000) and E3 (7000) are non-null; E2 and E4 are NULL $\rightarrow 2$.
  - `COUNT(DISTINCT DeptId)`: Distinct values are `{10, 20}` $\rightarrow 2$.
  - **Result:** `(4, 2, 2)`.
- **Part 2:**
  - `SUM(Bonus)` sums non-null values: $5000 + 7000 = 12000$.
  - `AVG(Bonus)` divides `SUM(Bonus)` by `COUNT(Bonus)`: $\frac{12000}{2} = 6000$. **(Fatal Trap: It does NOT divide by 4!)**
  - **Result:** `(12000, 6000.00)`.

Grouping Data: The GROUP BY Clause

The `GROUP BY` clause divides the rows of a relation into summary partitions (groups) based on matching values in one or more specified attributes. Aggregate functions are then computed independently for each partition:

1. Basic Grouping Syntax & Execution

```
SELECT DeptId, COUNT(*), AVG(Salary)
FROM EMPLOYEE
GROUP BY DeptId;
```

Execution Trace: The database engine scans `EMPLOYEE`, collects all tuples sharing the identical `DeptId` into distinct buckets, and applies `COUNT(*)` and `AVG(Salary)` to each bucket separately.

2. The Golden Invariant of GROUP BY

A fundamental syntactic rule enforced by relational database engines:

THE NON-AGGREGATE SELECT INVARIANT

Rule: Every column that appears in the `SELECT` list that is **not** enclosed inside an aggregate function **must** be included in the `GROUP BY` clause!

```
-- INVALID SYNTAX (Triggers SQL Error):
SELECT DeptId, EmpName, AVG(Salary) FROM EMPLOYEE GROUP BY DeptId;
```

Why? Because `DeptId` forms a single summary row per department, whereas `EmpName` contains multiple distinct names per department. The database cannot determine which single employee name to display alongside the summary average!

3. Multi-Column Grouping

Grouping can be performed across multiple attributes hierarchically:

```
SELECT DeptId, JobRole, COUNT(*), SUM(Salary)
FROM EMPLOYEE
GROUP BY DeptId, JobRole;
```

Semantics: Tuples are partitioned first by `DeptId`. Within each department, tuples are further subdivided by `JobRole`. An aggregate row is produced for each unique `(DeptId, JobRole)` combination.

4. Handling NULL Values in GROUP BY

If a column used in `GROUP BY` contains multiple rows with `NULL`:

- Standard SQL treats all `NULL` values as matching for grouping purposes.

- All rows having `NULL` in the grouping column are consolidated into **a single summary group**!

5. Grouping by Expressions & Date Parts

In practical database analytics, grouping is frequently performed on computed expressions or scalar functions rather than bare attribute names:

```
-- Count total registrations per year and month:
SELECT YEAR(RegDate) AS RegYear, MONTH(RegDate) AS RegMonth, COUNT(*) AS TotalStudents
FROM STUDENT
GROUP BY YEAR(RegDate), MONTH(RegDate)
ORDER BY RegYear DESC, RegMonth ASC;
```

MULTI-COLUMN GROUPING & AGGREGATE EVALUATION

Problem: Consider the relation `SALES(Region, Quarter, Amount)`:

- S1: ('North', 'Q1', 12000)
- S2: ('North', 'Q1', 18000)
- S3: ('North', 'Q2', 25000)
- S4: ('South', 'Q1', 15000)
- S5: ('South', 'Q2', 20000)
- S6: ('South', 'Q2', 22000)

Evaluate the output of:

```
SELECT Region, Quarter, SUM(Amount) AS TotalRevenue, COUNT(*) AS SalesCount
FROM SALES
GROUP BY Region, Quarter
ORDER BY Region ASC, Quarter ASC;
```

Solution:

- Group 1: `('North', 'Q1')` → S1 (12000) + S2 (18000) → `TotalRevenue = 30000`, `SalesCount = 2`.
- Group 2: `('North', 'Q2')` → S3 (25000) → `TotalRevenue = 25000`, `SalesCount = 1`.
- Group 3: `('South', 'Q1')` → S4 (15000) → `TotalRevenue = 15000`, `SalesCount = 1`.
- Group 4: `('South', 'Q2')` → S5 (20000) + S6 (22000) → `TotalRevenue = 42000`, `SalesCount = 2`.
- **Resulting Table:** 4 summary rows matching the distinct `(Region, Quarter)` tuples.

Filtering Grouped Records: The HAVING Clause

The `HAVING` clause specifies filter conditions applied to summary group partitions **after** the `GROUP BY` aggregation has been computed:

Comparing WHERE and HAVING

The distinction between `WHERE` and `HAVING` is one of the most critical topics in relational query processing:

Evaluation Criteria	WHERE Clause	HAVING Clause
Execution Stage	Evaluated at **Stage 2** (immediately after `FROM`).	Evaluated at **Stage 4** (after `GROUP BY` aggregation).
Target of Filtering	Filters individual base **rows** (tuples).	Filters summary **group partitions** (buckets).
Aggregate Functions	**Strictly Prohibited!** Cannot contain `COUNT()`, `SUM()`, `AVG()`.	**Permitted & Encouraged!** Designed specifically for aggregate conditions.
Primary Use Case	Prunes unwanted rows before grouping to optimize performance.	Discards groups failing summary threshold criteria.

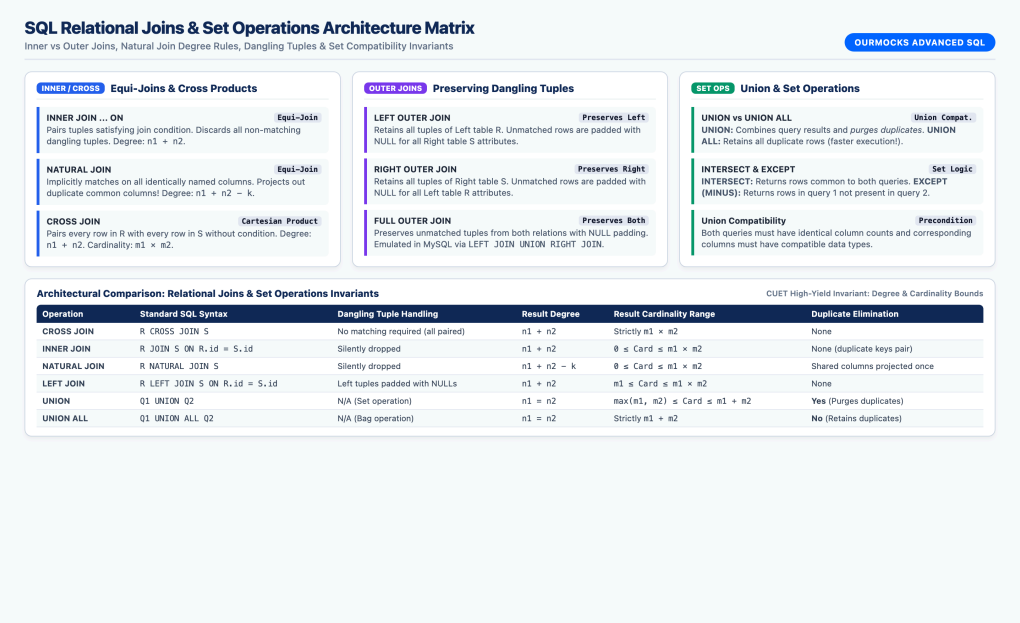


Figure 1: Relational Joins & Set Operations Architecture Matrix: Inner vs Outer Joins, Natural Join Degree Rules & Set Compatibility

Combining WHERE, GROUP BY, and HAVING

When all three clauses appear in a query, they execute in strict logical sequence:

```
SELECT DeptId, COUNT(*) AS TotalStaff, AVG(Salary) AS MeanPay
FROM EMPLOYEE
WHERE Status = 'Active'                -- Stage 2: Filter individual tuples
GROUP BY DeptId                        -- Stage 3: Partition surviving tuples
HAVING COUNT(*) >= 5                   -- Stage 4: Filter summary groups
ORDER BY MeanPay DESC;                 -- Stage 7: Sort final summary rows
```

TRACING WHERE VS HAVING EXECUTION

Problem: Consider the relation `ORDERS(OrderId, CustomerId, Amount, Status)`:

- O1: (1, 'C1', 400, 'Completed')
- O2: (2, 'C2', 600, 'Completed')
- O3: (3, 'C1', 300, 'Cancelled')
- O4: (4, 'C1', 500, 'Completed')
- O5: (5, 'C2', 200, 'Completed')

What is the output of the query:

```
SELECT CustomerId, SUM(Amount)
FROM ORDERS
WHERE Status = 'Completed'
GROUP BY CustomerId
HAVING SUM(Amount) > 700;
```

Solution:

1. **WHERE Filtering (`Status = 'Completed'`):** O3 is discarded. Surviving rows: O1 (C1, 400), O2 (C2, 600), O4 (C1, 500), O5 (C2, 200).
2. **GROUP BY Partitioning (`CustomerId`):**
   - Group 'C1': O1 (400) + O4 (500) → `SUM(Amount) = 900`.
   - Group 'C2': O2 (600) + O5 (200) → `SUM(Amount) = 800`.
3. **HAVING Filtering (`SUM(Amount) > 700`):**
   - Group 'C1': $900 > 700$ (Passes).
   - Group 'C2': $800 > 700$ (Passes).
4. **Final Output:**
   - `('C1', 900)`
   - `('C2', 800)`

Relational Joins in SQL

A **Join** combines attributes from two or more relations into a single composite result table based on related columns:

1. Cartesian Product (CROSS JOIN)

Pairs every tuple of the first relation with every tuple of the second relation:

```
SELECT * FROM STUDENT CROSS JOIN DEPARTMENT;
-- Equivalent comma syntax:
SELECT * FROM STUDENT, DEPARTMENT;
```

- Degree: $\text{Degree}(R \times S) = \text{Degree}(R) + \text{Degree}(S) = n_1 + n_2$.
- Cardinality: $\text{Cardinality}(R \times S) = \text{Cardinality}(R) \times \text{Cardinality}(S) = m_1 \times m_2$.

2. Equi-Join & INNER JOIN

An **Equi-Join** matches tuples between tables using exclusively the equality operator (=):

```
SELECT STUDENT.Name, DEPARTMENT.DeptName
FROM STUDENT
INNER JOIN DEPARTMENT ON STUDENT.DeptId = DEPARTMENT.DeptId;
```

- Only tuples that find a matching key in both tables are retained.
- **Dangling Tuples:** Rows in `STUDENT` without a matching `DeptId`, and departments without students, are silently discarded.

3. NATURAL JOIN

A `NATURAL JOIN` is an Equi-Join that automatically matches on **all identically named columns** between the two relations, with the critical property that **duplicate common columns are automatically projected out**:

```
SELECT * FROM STUDENT NATURAL JOIN DEPARTMENT;
```

- **Degree Formula:** $\text{Degree}(R \bowtie S) = n_1 + n_2 - k$ (where k is the count of shared attribute names).
- **Fatal Trap:** If two tables share column names that have different semantic meanings (e.g. `STUDENT.Name` and `DEPARTMENT.Name`), `NATURAL JOIN` will attempt to equate student names with department names, yielding an empty table! Always exercise caution with `NATURAL JOIN`.

4. Self Joins

A **Self Join** joins a table to itself by creating two distinct aliases (`rho`):

```
SELECT E.EmpName AS Employee, M.EmpName AS Manager
FROM EMPLOYEE E
JOIN EMPLOYEE M ON E.ManagerId = M.EmpId;
```

5. Non-Equi Joins & Multi-Table Joins

- **Non-Equi Joins:** Join conditions that use operators other than equality (`<`, `<=`, `>`, `>=`, `BETWEEN ... AND`):

```
-- Match student marks to letter grades without storing grades redundantly:
SELECT S.Name, S.Marks, G.Grade
FROM STUDENT S
JOIN GRADE_SCALE G ON S.Marks BETWEEN G.MinMarks AND G.MaxMarks;
```

- **Multi-Table Joins (Three or More Tables):** Relational engines evaluate multi-table queries by evaluating joins sequentially:

```
SELECT S.Name, C.CourseName, E.EnrollmentDate
FROM STUDENT S
JOIN ENROLLMENT E ON S.RollNo = E.RollNo
JOIN COURSE C ON E.CourseId = C.CourseId;
```

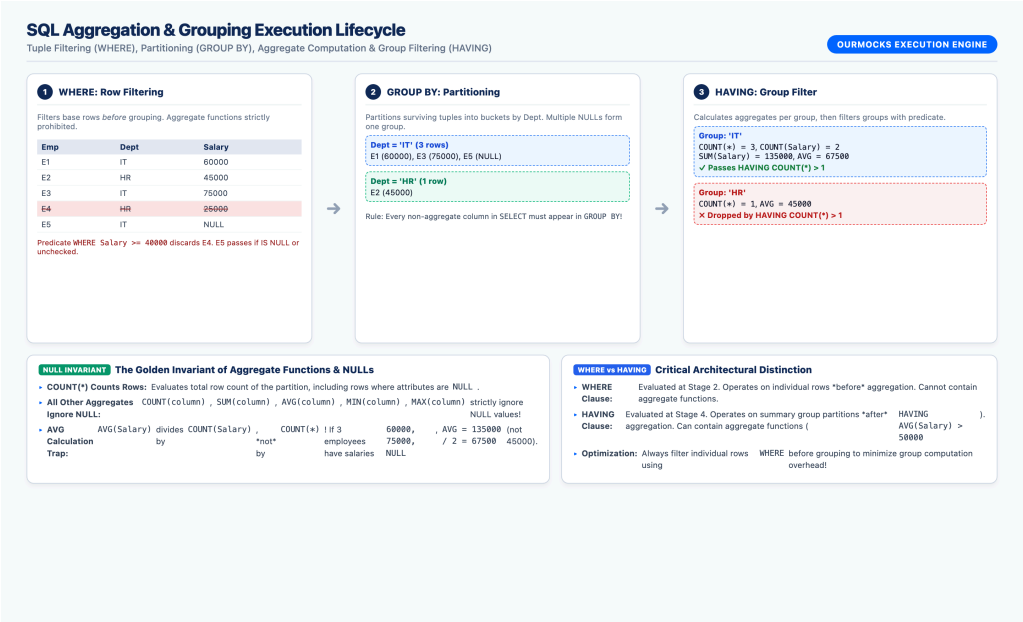


Figure 2: SQL Aggregation & Grouping Execution Lifecycle: Row Filtering, Partitioning, and Group Filtering Stages

Outer Joins in SQL

Standard inner joins discard tuples that fail to find a matching counterpart (dangling tuples). **Outer Joins** preserve these dangling tuples by populating the missing attributes with `NULL`:

1. LEFT OUTER JOIN (LEFT JOIN)

Preserves **all tuples from the left table**, regardless of whether a matching tuple exists in the right table:

```
SELECT S.RollNo, S.Name, D.DeptName
FROM STUDENT S
LEFT JOIN DEPARTMENT D ON S.DeptId = D.DeptId;
```

- If a student has `DeptId = NULL` or references a non-existent department, the student row is retained, and `DeptName` is displayed as `NULL`.
- Cardinality Bound: Cardinality(R LEFT JOIN S) $\geq$ Cardinality(R).

2. RIGHT OUTER JOIN (RIGHT JOIN)

Preserves **all tuples from the right table**. Unmatched attributes from the left table are filled with `NULL`:

```
SELECT S.Name, D.DeptName
FROM STUDENT S
RIGHT JOIN DEPARTMENT D ON S.DeptId = D.DeptId;
```

- Departments with zero enrolled students are preserved, displaying `Name` as `NULL`.

3. FULL OUTER JOIN (FULL JOIN)

Preserves unmatched tuples from **both** relations, padding missing values with `NULL`:

- Under standard SQL: `FROM R FULL OUTER JOIN S ON ...`
- In MySQL (which lacks native `FULL JOIN`), it is emulated via `UNION`:

```
SELECT S.Name, D.DeptName FROM STUDENT S LEFT JOIN DEPARTMENT D ON S.DeptId = D.DeptId
UNION
SELECT S.Name, D.DeptName FROM STUDENT S RIGHT JOIN DEPARTMENT D ON S.DeptId = D.DeptId;
```

4. Detecting Unmatched Records with Outer Joins

A classic SQL pattern to find orphan or unmatched records:

```
-- Find departments that currently have NO enrolled students:
SELECT D.DeptId, D.DeptName
FROM DEPARTMENT D
LEFT JOIN STUDENT S ON D.DeptId = S.DeptId
WHERE S.RollNo IS NULL;
```

OurMocks

INNER JOIN VS OUTER JOINS COMPARISON TRACE

Problem: Table `A(Id, Val)` has tuples `{(1, 'X'), (2, 'Y')}`. Table `B(Id, Descr)` has tuples `{(2, 'P'), (3, 'Q')}`. Evaluate the resulting rows and cardinality of:

1. `A INNER JOIN B ON A.Id = B.Id`
2. `A LEFT JOIN B ON A.Id = B.Id`
3. `A FULL OUTER JOIN B ON A.Id = B.Id`

Solution:

- **Inner Join:** Only `Id = 2` matches → `{(2, 'Y', 'P')}` (Cardinality = 1).
- **Left Join:** Preserves both rows of `A`. `Id = 1` gets `NULL` for `B.Descr` → `{(1, 'X', NULL), (2, 'Y', 'P')}` (Cardinality = 2).
- **Full Outer Join:** Preserves dangling rows from both sides → `{(1, 'X', NULL), (2, 'Y', 'P'), (NULL, NULL, 'Q')}` (Cardinality = 3).

Set Operations in SQL

Set operators combine the results of two independent `SELECT` queries into a single output relation:

Union Compatibility Precondition

Before any set operator can be evaluated, both queries must be **Union Compatible**:

1. Both queries must project the **exact same number of columns** (identical degree).
2. Corresponding columns from left to right must have **compatible data types**.

Set Theoretic Operators

1. `UNION`: Combines rows from both queries and **eliminates duplicate rows** by performing an internal sort.
2. `UNION ALL`: Concatenates rows from both queries **retaining all duplicates**. Much faster than `UNION` because no sorting or deduplication pass is executed!
3. `INTERSECT`: Returns only those rows that appear in **both** query result sets.
4. `EXCEPT` / `MINUS`: Returns rows present in the first query that do **not** appear in the second query.

UNION VS UNION ALL OUTPUT EVALUATION

Problem: Query 1 returns `{10, 20, 30}` (3 rows). Query 2 returns `{20, 30, 40, 50}` (4 rows).

1. What is the cardinality of `Query 1 UNION Query 2`?
2. What is the cardinality of `Query 1 UNION ALL Query 2`?

Solution:

- **Part 1 (UNION):** Eliminates duplicate values `{20, 30}`. The distinct set is `{10, 20, 30, 40, 50}`. Cardinality $= 5$.
- **Part 2 (UNION ALL):** Simply concatenates all rows without deduplication: $3 + 4 = 7$ rows. Cardinality $= 7$.

Nested Queries (Subqueries) in SQL

A **Subquery** (or **Nested Query**) is a `SELECT` statement enclosed inside parentheses and embedded within another SQL statement:

1. Single-Row Subqueries

Returns exactly one tuple with one column (a scalar value). Evaluated using standard relational comparison operators (`=, <, >, <=, >=, !=`):

```
-- Find employees who earn more than the company average salary:
SELECT EmpName, Salary
FROM EMPLOYEE
WHERE Salary > (SELECT AVG(Salary) FROM EMPLOYEE);
```

2. Multi-Row Subqueries

Returns multiple rows (a one-dimensional column vector). Must be evaluated using set membership and quantification operators:

- `IN`: True if the value matches **any** member in the subquery result set:
  ```
  SELECT EmpName FROM EMPLOYEE
  WHERE DeptId IN (SELECT DeptId FROM DEPARTMENT WHERE Location = 'Delhi');
  ```
- `NOT IN`: True if the value does not match any member in the subquery result. **(Caution: If the subquery result contains even a single `NULL`, `NOT IN` evaluates to `UNKNOWN` for all rows, returning zero results!)**
- `ALL`: Compares against every value returned by the subquery (`Salary > ALL (...)` means greater than the maximum).
- `ANY` / `SOME`: Compares against at least one value (`Salary > ANY (...)` means greater than the minimum).
- `EXISTS` / `NOT EXISTS`: Tests whether the subquery returns **at least one row** (boolean existence test).

3. Correlated Subqueries

A **Correlated Subquery** is an inner query that references one or more attributes from the outer query. Unlike independent subqueries (which execute once and pass a static result set), a correlated subquery must be **re-evaluated once for every candidate row** processed by the outer query:

```
-- Find employees who earn more than the average salary of their OWN department:
SELECT E1.EmpName, E1.DeptId, E1.Salary
FROM EMPLOYEE E1
WHERE E1.Salary > (
    SELECT AVG(E2.Salary)
    FROM EMPLOYEE E2
    WHERE E2.DeptId = E1.DeptId
);
```

4. Subqueries in the FROM Clause (Derived Tables)

When a subquery is placed in the `FROM` clause, it creates a temporary in-memory relation known as a **Derived Table** or **Inline View**. In standard SQL, a derived table **must always be assigned a table alias**:

```
-- Compute the maximum average department salary across all departments:
SELECT MAX(DeptMean) AS HighestMean
FROM (
    SELECT DeptId, AVG(Salary) AS DeptMean
    FROM EMPLOYEE
    GROUP BY DeptId
) AS DeptSummary;
```

FINDING THE SECOND HIGHEST SALARY USING PURE SQL

Problem: Write two standard SQL subquery solutions to retrieve the second highest salary from table `EMPLOYEE(EmpId, EmpName, Salary)` without using vendor-specific syntax like `LIMIT`:

1. Using the `MAX()` aggregate function in a single-row subquery.
2. Using a correlated subquery with `COUNT(DISTINCT ...)`.

Solution:

- **Method 1 (MAX Subquery):** Find the highest salary that is strictly smaller than the overall maximum salary:
```
SELECT MAX(Salary) AS SecondHighestSalary
FROM EMPLOYEE
WHERE Salary < (SELECT MAX(Salary) FROM EMPLOYEE);
```
- **Method 2 (Correlated Subquery for N-th Highest):** Find the salary where exactly 1 distinct salary is strictly greater:
```
SELECT E1.Salary
FROM EMPLOYEE E1
WHERE 1 = (
    SELECT COUNT(DISTINCT E2.Salary)
    FROM EMPLOYEE E2
    WHERE E2.Salary > E1.Salary
);
```

MASTER QUERY SYNTHESIS: JOINS, GROUPING & HAVING

Problem: Given relations:

- `CUSTOMER(CustId, CustName, City)`
- `ORDERS(OrderId, CustId, OrderDate, Amount)`

Write a query to retrieve the customer name and total amount spent for customers residing in `'Mumbai'` who have placed at least 2 orders and spent a cumulative amount exceeding 15000. Sort output by total amount in descending order.

Solution:
```
SELECT C.CustName, SUM(O.Amount) AS TotalSpent
FROM CUSTOMER C
JOIN ORDERS O ON C.CustId = O.CustId
WHERE C.City = 'Mumbai'
GROUP BY C.CustId, C.CustName
HAVING COUNT(O.OrderId) >= 2 AND SUM(O.Amount) > 15000
ORDER BY TotalSpent DESC;
```

- **Clause Pipeline Trace:**
  1. `FROM` & `JOIN`: Pairs matching customer and order tuples.
  2. `WHERE`: Discards non-Mumbai customers prior to aggregation.
  3. `GROUP BY`: Partitions surviving orders by customer.
  4. `HAVING`: Filters customer groups on aggregate order count (≥ 2) and total spent (> 15000).
  5. `SELECT` & `ORDER BY`: Projects customer names and sorts descending.

Database Views & Virtual Relations

A **View** is a virtual relation whose contents are dynamically derived from one or more underlying base tables via an executing SQL query. Unlike physical tables, a view stores **no data of its own on disk**; instead, the database catalog stores only the compiled `SELECT` query definition:

1. Creating and Dropping Views

```
CREATE VIEW ViewName [(ColumnAliasList)] AS
SELECT AttributeList
FROM BaseTable
WHERE PredicateCondition
[WITH CHECK OPTION];
```

- **Dropping a View:** `DROP VIEW ViewName;` removes the view definition from the catalog without deleting any data from the underlying base tables.

2. Advantages of Views in Database Architecture

1. **Data Security & Column Masking:** Restricts user access to specific sensitive columns (e.g., exposing employee names and departments while hiding salary figures).
2. **Query Simplicity & Abstraction:** Encapsulates complex multi-table joins, outer joins, and aggregations into a single reusable virtual table.
3. **Logical Data Independence:** Insulates client applications from structural changes in underlying base tables (e.g., splitting a table into two normalized tables while reconstructing the original schema via a view).

3. Updatable Views vs Read-Only Views

A view is considered **updatable** (allowing `INSERT`, `UPDATE`, `DELETE` operations that propagate to base tables) if and only if every row in the view maps uniquely to a single identifiable row in a single base table.

Feature / Operation	Impact on View Updatability
`DISTINCT` Keyword	Makes view strictly **Read-Only** (rows are deduplicated; 1:1 mapping lost).
Aggregate Functions (`SUM`, `AVG`)	Makes view strictly **Read-Only** (summary values cannot be mapped back to base rows).
`GROUP BY` or `HAVING`	Makes view strictly **Read-Only** (represents summary buckets).
Multi-Table Joins	Generally Read-Only under standard SQL (updates cannot span multiple tables).
`WITH CHECK OPTION`	Ensures that any row inserted or updated via the view satisfies the view's `WHERE` clause!

VIEW CREATION AND WITH CHECK OPTION

Problem: Consider the relation `EMPLOYEE(EmpId, EmpName, DeptId, Salary)`.

1. Create a view `IT_STAFF` containing employees in department `10` whose salary exceeds `50000`. Ensure that no user can insert an employee through this view with `Salary <= 50000` or `DeptId != 10`.
2. What happens if a user executes: `INSERT INTO IT_STAFF VALUES (205, 'Manish', 10, 45000);`?

Solution:

- **Part 1:**
```
CREATE VIEW IT_STAFF AS
SELECT EmpId, EmpName, DeptId, Salary
FROM EMPLOYEE
WHERE DeptId = 10 AND Salary > 50000
WITH CHECK OPTION;
```
- **Part 2:** Because `WITH CHECK OPTION` is declared, the database evaluates the view predicate before executing the insert. Since `Salary = 45000` violates the condition `Salary > 50000`, the insert statement is rejected with a check option violation error!

OurMocks

Master Synthesis Mindmap: Advanced SQL — Grouping, Joins & Set Operations

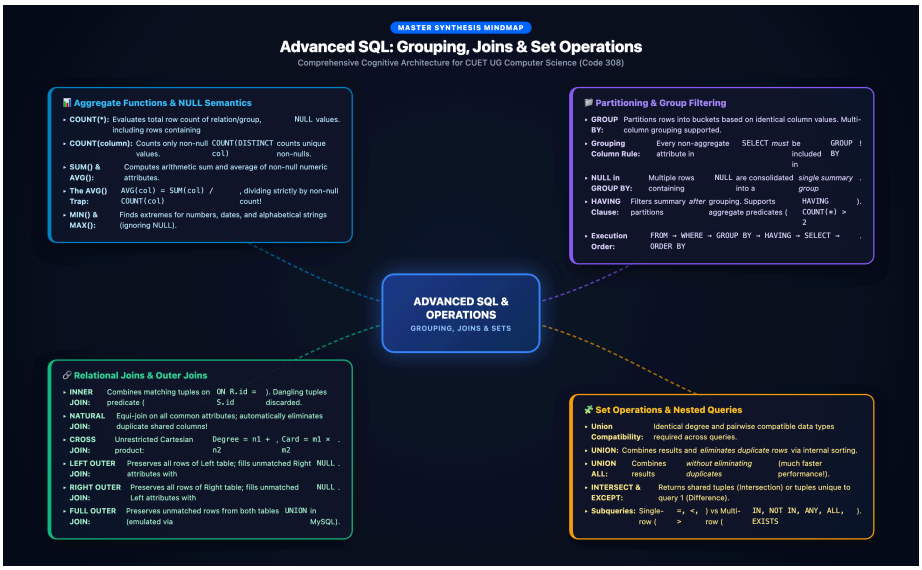


Figure 3: Master Synthesis Mindmap: Aggregate Functions, Grouping Mechanics, Joins Hierarchy & Set Operations

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

RAPID REVISION CHEATSHEET: ADVANCED SQL COMMANDS & OPERATIONS

- **Aggregate Functions:** `COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`.
- **The NULL Invariant:** `COUNT(*)` counts all rows (including NULLs). All other aggregates ignore NULLs!
- **AVG Formula:** `AVG(col) = SUM(col) / COUNT(col)` (divides strictly by count of non-null values).
- **GROUP BY:** Partitions rows by attribute(s). Multiple NULLs form a single group. All non-aggregate `SELECT` columns must be in `GROUP BY`.
- **WHERE vs HAVING:** `WHERE` filters rows before grouping (no aggregates!). `HAVING` filters summary groups after grouping (aggregates allowed!).
- **CROSS JOIN:** Cartesian product ($n_1 + n_2$ columns, $m_1 \times m_2$ rows).
- **NATURAL JOIN:** Equi-join on all shared columns. Duplicate common columns are projected out ($n_1 + n_2 - k$ columns).
- **LEFT JOIN:** Preserves all left table rows. Unmatched right table attributes are padded with `NULL`.
- **RIGHT JOIN:** Preserves all right table rows. Unmatched left table attributes are padded with `NULL`.
- **FULL OUTER JOIN:** Preserves all rows from both tables with NULL padding (emulated via `UNION` in MySQL).
- **UNION vs UNION ALL:** `UNION` removes duplicate rows (requires sorting). `UNION ALL` preserves all duplicates (faster!).
- **Union Compatibility:** Both queries must have identical column count and compatible data types.
- **Subqueries:** Single-row (`=, <, >`), Multi-row (`IN, NOT IN, ANY, ALL, EXISTS`).

THE SEVEN FATAL CUET TRAPS IN ADVANCED SQL

1. **The `COUNT(*)` vs `COUNT(col)` Trap:** `COUNT(*)` counts total rows. `COUNT(col)` counts only rows where `col IS NOT NULL`. If a table has 10 rows and 3 have `col = NULL`, `COUNT(*) = 10` while `COUNT(col) = 7`!
2. **The Aggregate AVG() Divisor Trap:** When computing `AVG(col)`, the denominator is `COUNT(col)`, **not** `COUNT(*)`. A table with values `(10, 20, NULL)` has `AVG = (10 + 20) / 2 = 15`, **not** $\frac{30}{3} = 10$!
3. **The Aggregate Function in WHERE Trap:** Writing `WHERE AVG(Salary) > 50000` or `WHERE COUNT(*) >= 2` produces a fatal SQL compilation error! Aggregate conditions must be placed in `HAVING`.
4. **The Non-Aggregate SELECT Column Trap:** Including an attribute in `SELECT` that is not part of `GROUP BY` and not enclosed in an aggregate function triggers an error in standard SQL (e.g. `SELECT DeptId, EmpName, AVG(Salary) ... GROUP BY DeptId`).
5. **The Natural Join Semantic Collision Trap:** If two tables have columns with identical names but different meanings (e.g. `STUDENT.Name` and `COURSE.Name`), `NATURAL JOIN` equates `STUDENT.Name = COURSE.Name`, returning zero rows!
6. **The NOT IN with Subquery NULL Trap:** If a subquery used with `NOT IN` returns even a single `NULL` value, the condition `x NOT IN (..., NULL)` evaluates to `UNKNOWN` for all rows, resulting in **zero rows returned**!
7. **The UNION ALL Duplication Trap:** `UNION ALL` does not eliminate duplicate rows between queries or within individual queries. If Query 1 returns 5 rows and Query 2 returns 5 rows, `UNION ALL` unconditionally returns exactly 10 rows.

CUET Focused Multiple Choice Questions (Part 1)

Q1. A table `EMPLOYEE` contains 6 records where the `Commission` column holds values: `1000, 2000, NULL, 3000, NULL, 4000`. What are the exact values returned by the query `SELECT COUNT(*), COUNT(Commission), AVG(Commission) FROM EMPLOYEE;`?

(A) `6, 4, 2500` **(B)** `6, 6, 1666.67`
(C) `4, 4, 2500` **(D)** `6, 4, 1666.67`

Sol. Q1. `COUNT(*)` counts all 6 tuples in the table regardless of NULLs. `COUNT(Commission)` counts only the 4 non-null values. `AVG(Commission)` calculates $\frac{1000+2000+3000+4000}{4} = \frac{10000}{4} = 2500$, dividing strictly by the count of non-null values (4) rather than total rows (6).
Answer: (A)

Q2. Which clause in SQL is evaluated after the `GROUP BY` clause to filter out summary group partitions based on an aggregate condition?

(A) `WHERE` **(B)** `HAVING`
(C) `ORDER BY` **(D)** `SELECT`

Sol. Q2. The `HAVING` clause is specifically designed to filter summary group partitions **after** grouping has taken place. `WHERE` filters individual tuples **before** grouping and cannot contain aggregate functions.
Answer: (B)

Q3. Consider relation R with Degree = 4 and Cardinality = 10, and relation S with Degree = 3 and Cardinality = 5. If R and S share exactly 2 common attribute names, what is the Degree of the Natural Join $R \bowtie S$?

(A) 7 **(B)** 6
(C) 5 **(D)** 2

Sol. Q3. The degree of a Natural Join is given by $\text{Degree}(R \bowtie S) = \text{Degree}(R) + \text{Degree}(S) - k$, where k is the number of shared attributes. Here, Degree $= 4 + 3 - 2 = 5$. Duplicate common columns are projected out once.
Answer: (C)

Q4. Why does the following SQL query fail with an error when executed on a relational database? `SELECT DeptId, EmpName, MAX(Salary) FROM EMPLOYEE GROUP BY DeptId;`

(A) The `MAX()` aggregate function cannot be used with `GROUP BY`
(B) The keyword `ORDER BY` is mandatory whenever `GROUP BY` is used
(C) `DeptId` cannot be placed in the `SELECT` list
(D) `EmpName` is not enclosed in an aggregate function and is not part of the `GROUP BY` clause

Sol. Q4. Under standard SQL, every column in the `SELECT` list that is not enclosed inside an aggregate function must appear in the `GROUP BY` clause. Because `EmpName` has multiple distinct values per `DeptId` partition, the engine cannot select a single value to display alongside `MAX(Salary)`.
Answer: (D)

Q5. Query 1 returns 8 rows and Query 2 returns 6 rows. If 2 rows are common to both result sets, what is the total number of rows returned by `Query 1 UNION Query 2`?

(A) 12 **(B)** 14
(C) 10 **(D)** 8

Sol. Q5. The `UNION` operator combines result sets and eliminates duplicate rows. By the Principle of Inclusion-Exclusion: Total Rows $= 8 + 6 - 2 = 12$. (In contrast, `UNION ALL` would return $8 + 6 = 14$ rows).
Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. In a `LEFT OUTER JOIN` between table `STUDENT` (left) and table `DEPARTMENT` (right), what appears in the department columns for a student whose `DeptId` is NULL?

(A) The student record is discarded from the result
(B) The department columns are populated with `NULL`
(C) The database displays the default department name
(D) A referential integrity runtime error is generated

Sol. Q6. A `LEFT JOIN` preserves all tuples from the left table. For tuples that fail to find a matching partner in the right table (such as an unassigned or NULL department ID), the right table attributes are padded with `NULL`.
Answer: (B)

Q7. What is the effect of executing `SELECT DeptId, COUNT(*) FROM EMP GROUP BY DeptId;` when several employees have `DeptId = NULL`?

(A) All employees with `DeptId = NULL` are excluded from the result
(B) Each NULL employee forms an individual separate single-row group
(C) All employees with `DeptId = NULL` are gathered into a single summary group
(D) The query triggers a divide-by-zero database exception

Sol. Q7. Under standard SQL grouping semantics, `NULL` values are treated as matching each other. Consequently, all tuples having `DeptId = NULL` are consolidated into a single summary partition displaying `DeptId` as `NULL`.
Answer: (C)

Q8. Which operator is used with a multi-row subquery to test whether a value is greater than every single value returned by the subquery?

(A) `> ANY`
(B) `> SOME`
(C) `IN`
(D) `> ALL`

Sol. Q8. The `> ALL` operator compares a scalar value against every value returned by the subquery. The condition is satisfied if and only if the value is strictly greater than the maximum value in the subquery result set.
Answer: (D)

Q9. A user wants to find departments that have more than 10 employees. Which of the following queries expresses this requirement correctly?

(A) `SELECT DeptId, COUNT(*) FROM EMP GROUP BY DeptId HAVING COUNT(*) > 10;`
(B) `SELECT DeptId, COUNT(*) FROM EMP WHERE COUNT(*) > 10 GROUP BY DeptId;`
(C) `SELECT DeptId, COUNT(*) FROM EMP GROUP BY DeptId WHERE COUNT(*) > 10;`
(D) `SELECT DeptId FROM EMP HAVING COUNT(DeptId) > 10;`

Sol. Q9. Filtering on the result of an aggregate function (`COUNT(*) > 10`) must be performed using the `HAVING` clause placed after `GROUP BY`. Aggregate functions are strictly forbidden inside the `WHERE` clause.
Answer: (A)

Q10. Consider the two tables `R(A, B)` containing 4 rows and `S(C, D)` containing 5 rows. What is the Cardinality and Degree of the Cartesian product `SELECT * FROM R, S;`?

(A) Cardinality = 9, Degree = 4
(B) Cardinality = 20, Degree = 4
(C) Cardinality = 20, Degree = 2
(D) Cardinality = 9, Degree = 20

Sol. Q10. In a Cartesian Product $R \times S$:

- Degree is additive: Degree $= 2 + 2 = 4$.
- Cardinality is multiplicative: Cardinality $= 4 \times 5 = 20$.

Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. Which SQL join produces a result set containing only tuples that satisfy the join predicate, completely discarding all dangling tuples from both tables?

(A) `FULL OUTER JOIN` (B) `LEFT OUTER JOIN`
(C) `INNER JOIN` (D) `RIGHT OUTER JOIN`

Sol. Q11. An `INNER JOIN` retains only those pairs of tuples that satisfy the join predicate (e.g. `ON R.id = S.id`). Any tuple that does not have a matching partner in the opposite table is discarded.
Answer: (C)

Q12. What happens when the query `SELECT * FROM CUSTOMER WHERE CustId NOT IN (SELECT ReferrerId FROM REFERRALS);` is executed if `REFERRALS` contains at least one record where `ReferrerId IS NULL`?

(A) It correctly returns all customers who were not referred (B) It throws a NULL pointer syntax exception
(C) It ignores the NULL record and evaluates the remaining referrers (D) It returns an empty result set (0 rows) because comparison with NULL yields UNKNOWN

Sol. Q12. The expression `x NOT IN (1, 2, NULL)` expands to `(x != 1) AND (x != 2) AND (x != NULL)`. Since `x != NULL` evaluates to `UNKNOWN`, the entire `AND` conjunction evaluates to `UNKNOWN` or `FALSE`. Consequently, no rows can ever satisfy the condition, returning an empty set.
Answer: (D)

Q13. Which set operator combines two queries while preserving all duplicate rows without executing an internal deduplication sort pass?

(A) `UNION ALL` (B) `UNION`
(C) `INTERSECT` (D) `MINUS`

Sol. Q13. `UNION ALL` concatenates the result sets of two union-compatible queries directly without eliminating duplicate rows. Because it does not perform an internal sort or hash pass for deduplication, it executes substantially faster than `UNION`.
Answer: (A)

Q14. Consider a query joining `EMPLOYEE E` with itself: `FROM EMPLOYEE E JOIN EMPLOYEE M ON E.MgrId = M.EmpId`. What is this join pattern formally called?

(A) Cross Join (B) Self Join
(C) Natural Join (D) Outer Union

Sol. Q14. A `Self Join` is a relational join in which a table is joined with itself using distinct table aliases (here `E` for employee and `M` for manager) to model hierarchical relationships such as employee-supervisor reporting structures.
Answer: (B)

Q15. What is the output of `SELECT SUM(Commission) FROM SALES;` if every single record in the `SALES` table currently has `Commission = NULL`?

(A) `0` (B) `0.00`
(C) `NULL` (D) A runtime runtime exception

Sol. Q15. Under ANSI SQL standards, if an aggregate function (such as `SUM()`, `AVG()`, `MIN()`, or `MAX()`) is evaluated on a column where all rows in the group are `NULL`, the function evaluates to `NULL` (not `0`).
Answer: (C)